quectel-BLE

版本：1.0

日期：2026-03-26

状态：受控/临时文件

文件保密级别：（请勾选 ■ ）		
绝密 □	保密 □	公开 ■

文件管控表

文件变更记录			
修订日期	最新版本	修订内容	编制
2026-04-02	1	首次发行	Wells Li

上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼　　邮编：200233
电话：+86 21 5108 6236　　邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：http://www.quectel.com/cn/support/sales.htm。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
http://www.quectel.com/cn/support/technical.htm 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬软件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2024，保留一切权利。
Copyright © Quectel Wireless Solutions Co., Ltd. 2024.

目录

1 模块概述 7

2 BLE 管理 7

2.1. BLE 7

2.1.1. 接口概述 7

2.1.2. 函数原型 7

2.1.3. 参数描述 7

2.1.4. 返回值描述 7

2.1.5. 示例 8

2.2. init 8

2.2.1. 接口概述 8

2.2.2. 函数原型 8

2.2.3. 参数描述 8

2.2.4. 返回值描述 8

2.2.5. 示例 8

2.3. deinit 9

2.3.1. 接口概述 9

2.3.2. 函数原型 9

2.3.3. 参数描述 9

2.3.4. 返回值描述 9

2.3.5. 示例 9

3 BLE 基础控制 9

3.1. start 9

3.1.1. 接口概述 9

3.1.2. 函数原型 9

3.1.3. 参数描述 10

3.1.4. 返回值描述 10

3.1.5. 示例 10

3.2. stop 10

3.2.1. 接口概述 10

3.2.2. 函数原型 10

3.2.3. 参数描述 10

3.2.4. 返回值描述 10

3.2.5. 示例 10

3.3. set_dataformat 10

3.3.1. 接口概述 10

3.3.2. 函数原型 11

3.3.3. 参数描述 11

3.3.4. 返回值描述 11

3.3.5. 示例 11

3.4. get_addr 11

3.4.1. 接口概述 11

3.4.2. 函数原型 11
3.4.3. 参数描述 11
3.4.4. 返回值描述 11
3.4.5. 示例 11

4 广播操作 12
4.1. set_adv_params 12
4.1.1. 接口概述 12
4.1.2. 函数原型 12
4.1.3. 参数描述 12
4.1.4. 返回值描述 12
4.1.5. 示例 12
4.2. set_adv_data 13
4.2.1. 接口概述 13
4.2.2. 函数原型 13
4.2.3. 参数描述 13
4.2.4. 返回值描述 13
4.2.5. 示例 13
4.3. set_scan_rsp_data 13
4.3.1. 接口概述 13
4.3.2. 函数原型 13
4.3.3. 参数描述 14
4.3.4. 返回值描述 14
4.3.5. 示例 14
4.4. advertise 14
4.4.1. 接口概述 14
4.4.2. 函数原型 14
4.4.3. 参数描述 14
4.4.4. 返回值描述 14
4.4.5. 示例 14

5 GATT 数据库操作 15
5.1. add_service 15
5.1.1. 接口概述 15
5.1.2. 函数原型 15
5.1.3. 参数描述 15
5.1.4. 返回值描述 15
5.1.5. 示例 15
5.2. add_character 15
5.2.1. 接口概述 15
5.2.2. 函数原型 15
5.2.3. 参数描述 15
5.2.4. 返回值描述 16
5.2.5. 示例 16
5.3. set_character_value 16

5.3.1. 接口概述 16
5.3.2. 函数原型 16
5.3.3. 参数描述 16
5.3.4. 返回值描述 16
5.3.5. 示例 16
5.4. add_descriptor 17
5.4.1. 接口概述 17
5.4.2. 函数原型 17
5.4.3. 参数描述 17
5.4.4. 返回值描述 17
5.4.5. 示例 17

6 连接数据交互 17
6.1. notify 17
6.1.1. 接口概述 17
6.1.2. 函数原型 18
6.1.3. 参数描述 18
6.1.4. 返回值描述 18
6.1.5. 使用说明 18
6.1.6. 示例 18
6.2. indicate 18
6.2.1. 接口概述 18
6.2.2. 函数原型 18
6.2.3. 参数描述 19
6.2.4. 返回值描述 19
6.2.5. 使用说明 19
6.2.6. 示例 19

7 MTU 与连接参数操作 19
7.1. exchange_mtu 19
7.1.1. 接口概述 19
7.1.2. 函数原型 20
7.1.3. 参数描述 20
7.1.4. 返回值描述 20
7.1.5. 示例 20

8 回调事件说明 20
8.1. 连接事件 20
8.1.1. 事件类型 20
8.1.2. 回调数据格式 20
8.1.3. 示例 21
8.2. MTU 事件 21
8.2.1. 事件类型 21
8.2.2. 回调数据格式 21
8.2.3. 示例 21
8.3. Characteristic 数据事件 21

8.3.1. 事件类型 21
8.3.2. 回调数据格式 21
8.3.3. 字段说明 22
8.3.4. 示例 22
8.4. Descriptor 数据事件 22
8.4.1. 事件类型 22
8.4.2. 回调数据格式 22
8.4.3. 字段说明 22
8.4.4. 示例 23
8.5. 回调完整示例 23

9 常量定义 23
9.1. 广播类型常量 23
9.2. 地址类型常量 24
9.3. Character 属性常量 24
9.4. 权限常量 24
9.5. 数据格式常量 25
9.6. 事件常量 25

1 模块概述

BLE 模块基于 BLE 4.2 协议，提供 BLE GATT Server （从机）相关功能，包括：

- BLE 名称设置与广播参数配置本地音频文件播放
- 广播数据与扫描响应数据配置
- GATT Service / Characteristic / Descriptor 的创建
- Notify / Indicate 数据发送
- BLE 事件回调通知

该模块基于底层 BLE 接口进行封装，并以 MicroPython 对象方式对外提供统一接口，便于用户在 Python 脚本中进行 BLE 外设功能开发。

2 BLE 管理

2.1. BLE

2.1.1. 接口概述

创建一个新的 BLE 管理器对象。

2.1.2. 函数原型

ble = quectel.BLE()

2.1.3. 参数描述

无。

2.1.4. 返回值描述

返回一个新的 BLE 对象实例。

2.1.5. 示例

```
import quectel
ble = quectel.BLE()
```

2.2. init

2.2.1. 接口概述

初始化 Audio 管理器，并可选择注册音频事件回调函数。

2.2.2. 函数原型

```
result = ble.init(callback)
```

2.2.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
callback	否	function	None	BLE 事件回调函数，接收 1 个参数：event_dict

2.2.4. 返回值描述

- 初始化成功返回 True
- 初始化失败返回 False
- 若 callback 不是可调用对象且不为 None，则抛出异常。

2.2.5. 示例

```
import quectel

def ble_cb(event):
    print("BLE event:", event)

ble = quectel.BLE()
if ble.init(ble_cb):
    print("BLE 初始化成功")
else:
    print("BLE 初始化失败")
```

2.3. deinit

2.3.1. 接口概述

反初始化 BLE 管理器并释放资源。

2.3.2. 函数原型

```
ble.deinit()
```

2.3.3. 参数描述

无

2.3.4. 返回值描述

无返回值，若 BLE 未初始化，则抛出异常。

2.3.5. 示例

```
import quectel

ble = quectel.BLE()
ble.init()
ble.deinit()
```

3 BLE 基础控制

3.1. start

3.1.1. 接口概述

启动 BLE Server，设置设备名称及默认广播参数，并生成默认广播数据。
说明：
调用本接口后仅完成 BLE 功能开启及广播参数/广播数据配置，不会自动开始广播。
若需开始广播，还需继续调用 ble.advertise()。

3.1.2. 函数原型

```
ble.start(name)
```

3.1.3. 参数描述

参数名	是 否必填	数据类型	默认值	说明
name	是	str	-	BLE 设备名称

3.1.4. 返回值描述

无返回值。 若执行失败，则抛出异常。。

3.1.5. 示例

```
import quectel

ble = quectel.BLE()
ble.init()
ble.start("Quectel_BLE")
```

3.2. stop

3.2.1. 接口概述

关闭 BLE Server。

3.2.2. 函数原型

```
ble.stop()
```

3.2.3. 参数描述

无。

3.2.4. 返回值描述

无返回值，若失败抛出异常。

3.2.5. 示例

```
ble.stop()
```

3.3. set_dataformat

3.3.1. 接口概述

设置 BLE 通知/指示的数据格式，如果不调用默认是 HEX 模式。

3.3.2. 函数原型

ble.set_dataformat(fmt)

3.3.3. 参数描述

参数名	是 否必填	数据类型	默认值	说明
fmt	是	int	-	数据格式，参考 9.5 数据格式常量

3.3.4. 返回值描述

无返回值。 若参数非法或执行失败，则抛出异常。

3.3.5. 示例

```
import quectel

ble = quectel.BLE()
ble.init()
ble.set_dataformat(quectel.BLE.DATAFMT_STRING)
```

3.4. get_addr

3.4.1. 接口概述

获取当前 BLE 地址。

3.4.2. 函数原型

addr = ble.get_addr()

3.4.3. 参数描述

无。

3.4.4. 返回值描述

返回 BLE 地址字符串。 若获取失败，返回 None。

3.4.5. 示例

```
addr = ble.get_addr()
print("BLE 地址:", addr)
```

4 广播操作

4.1. set_adv_params

4.1.1. 接口概述

设置 BLE 广播参数。ble start 内部会默认设置
(128, 160,QL_BLE_ADV_CONNECTABLE_UNDIRECTED,
QL_BLE_ADDR_RANDOM,7);

4.1.2. 函数原型

ble.set_adv_params(min_interval, max_interval, adv_type, own_addrtype, channel_map)

4.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
min_interval	是	int	-	最小广播时间间隔。需小于。范围：32~16384；间隔为 0.625 毫秒；广播间隔时间范围：20 毫秒~10.24 秒。
max_interval	是	int	-	最大广播时间间隔。范围：32~16384；间隔为 0.625 毫秒；广播间隔时间范 围：20 毫秒~10.24 秒
adv_type	是	int	-	广播类型，参考 9.1 广播类型常量
own_addrtype	是	int		本机地址类型，参考 9.2 地址类型常量
channel_map	是	int		发送该广播报文的信道 1 广播信道 37 2 广播信道 38 3 广播信道 37、38 4 广播信道 39 5 广播信道 37、39 6 广播信道 38、39 7 广播信道 37、38、39

4.1.4. 返回值描述

无返回值。 若参数非法或执行失败，则抛出异常。

4.1.5. 示例

import quectel

```
ble = quectel.BLE()
ble.init()

ble.set_adv_params(128,160,
    quectel.BLE.ADV_CONNECTABLE_UNDIRECTED,
    quectel.BLE.ADDR_RANDOM,7)
```

4.2. set_adv_data

4.2.1. 接口概述

设置广播数据。ble start 内部会默认设置
FLAGS+BELNAME

4.2.2. 函数原型

ble.set_adv_data(hex_data, adv_len_bytes)

4.2.3. 参数描述

参数名	是 否必填	数据类型	默认值	说明
hex_data	是	str	-	广播数据，必须为十六进制字符串
adv_len_bytes	否	int	自动计算	广播数据字节长度

4.2.4. 返回值描述

无返回值。若 hex_data 不是合法十六进制字符串、长度超过 31 字节或长度不匹配，则抛出异常。

4.2.5. 示例

```
# Flags + Complete Local Name("TEST")
adv_hex = "020106050954455354"
ble.set_adv_data(adv_hex)
```

4.3. set_scan_rsp_data

4.3.1. 接口概述

设置扫描响应数据。主要用于**补充广播数据中放不下的信息。**

4.3.2. 函数原型

ble.set_scan_rsp_data(hex_data, rsp_len_bytes)

4.3.3. 参数描述

参数名	是 否必填	数据类型	默认值	说明
hex_data	是	str	-	扫描响应数据，必须为十六进制字符串
rsp_len_bytes	否	int	自动计算	扫描响应数据字节长度

4.3.4. 返回值描述

无返回值。若 hex_data 不是合法十六进制字符串、长度超过 31 字节或长度不匹配，则抛出异常。

4.3.5. 示例

```
rsp_hex = "04FFE00301"
ble.set_scan_rsp_data(rsp_hex)
```

4.4. advertise

4.4.1. 接口概述

完成 GATT 服务注册，并使能广播。

4.4.2. 函数原型

```
ble.advertise()
```

4.4.3. 参数描述

无。

4.4.4. 返回值描述

无返回值。
若执行失败，则抛出异常。

4.4.5. 示例

```
ble.advertise()
```

5 GATT 数据库操作

5.1. add_service

5.1.1. 接口概述

添加一个 GATT Service。

5.1.2. 函数原型

ble.add_service(servID, uuid, primary)

5.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
servID	是	int	-	Service ID
uuid	是	int	-	16-bit UUID
primary	是	int	-	是否为主服务，True 表示主服务

5.1.4. 返回值描述

无返回值。 若执行失败，则抛出异常。

5.1.5. 示例

ble.add_service(0, 0xFFF0, True)

5.2. add_character

5.2.1. 接口概述

向指定 Service 添加一个 Characteristic，要保证 UUID 唯一。

5.2.2. 函数原型

ble.add_character(servID, charaID, properties, uuid)

5.2.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
servID	是	int	-	Service ID

charaID	是	int	-	Characteristic ID
properties	是	int	-	Characteristic 属性，参考 9.3 Character 属性常量
uuid	是	int		16-bit UUID

5.2.4. 返回值描述

无返回值。 若执行失败，则抛出异常。

5.2.5. 示例

ble.add_character(0,0,quectel.BLE.PROP_READ | quectel.BLE.PROP_NOTIFY, 0xFFF1)

5.3. set_character_value

5.3.1. 接口概述

设置 Characteristic 的初始值及权限。

5.3.2. 函数原型

ble.set_character_value(servID, charaID, permission, uuid, value_len, value_hex)

5.3.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
servID	是	int	-	Service ID
charaID	是	int	-	Characteristic ID
permission	是	int	-	访问权限，参考 9.4 权限常量
uuid	是	int	-	16-bit UUID
value_len	是	int	-	Characteristic 值长度（字节）
value_hex	是	str	-	Characteristic 初始值，十六进制字符串

5.3.4. 返回值描述

无返回值。若 value_hex 不是合法十六进制字符串或执行失败，则抛出异常。

5.3.5. 示例

ble.set_character_value(0,0,BLE.PERM_READ | BLE.PERM_WRITE, 0xFFF1,244,“1234”)

5.4. add_descriptor

5.4.1. 接口概述

向指定 Characteristic 添加一个 Descriptor。

5.4.2. 函数原型

ble.add_descriptor(servID, charaID, permission, uuid, value_hex)

5.4.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
servID	是	int	-	Service ID
charaID	是	int	-	Characteristic ID
permission	是	int	-	访问权限，参考 9.4 权限常量
uuid	是	int	-	16-bit UUID
value_hex	是	str	-	Descriptor 初始值，十六进制字符串

5.4.4. 返回值描述

无返回值。若 value_hex 不是合法十六进制字符串或执行失败，则抛出异常。

5.4.5. 示例

ble.add_descriptor(0,0,quectel.BLE.PERM_READ|quectel.BLE.PERM_WRITE, 0x2902, "0000")

6 连接数据交互

6.1. notify

6.1.1. 接口概述

通过指定 Characteristic UUID 向对端发送 Notify 数据。

6.1.2. 函数原型

ble.notify(uuid, length, value)

6.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
uuid	是	int	-	Characteristic UUID
length	是	int	-	要发送的实际数据长度（字节数） 根据 set_dataformat 设置的格式确定。
value	是	str	-	发送内容

6.1.4. 返回值描述

无返回值。 若 value 为空或执行失败，则抛出异常。

6.1.5. 使用说明

该接口当前接收的 value 为字符串类型。
实际发送内容应结合 set_dataformat() 配置的格式使用：

当数据格式为 DATAFMT_STRING 时，value 应为普通字符串，length 通常为字符串长度
当数据格式为 DATAFMT_HEX 时，value 应为十六进制字符串，length 应为实际字节长度而不是字符串长度。

6.1.6. 示例

```
# 在合适的地方，只需要设置一次
ble.set_dataformat(quectel.BLE.DATAFMT_STRING)

msg = "25.3"
ble.notify(0xFFF1, len(msg), msg)
```

6.2. indicate

6.2.1. 接口概述

通过指定 Characteristic UUID 向对端发送 Indicate 数据。

6.2.2. 函数原型

ble.indicate(uuid, length, value)

6.2.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
uuid	是	int	-	Characteristic UUID
length	是	int	-	要发送的实际数据长度（字节数） 根据 set_dataformat 设置的格式确定。
value	是	str	-	发送内容

6.2.4. 返回值描述

无返回值。 若 value 为空或执行失败，则抛出异常。

6.2.5. 使用说明

该接口当前接收的 value 为字符串类型。
实际发送内容应结合 set_dataformat() 配置的格式使用：

当数据格式为 DATAFMT_STRING 时，value 应为普通字符串，length 通常为字符串长度
当数据格式为 DATAFMT_HEX 时，value 应为十六进制字符串，length 应为实际字节长度而不是字符串长度。

6.2.6. 示例

```
# 在合适的地方，只需要设置一次
ble.set_dataformat(quectel.BLE.DATAFMT_HEX)

# 25.3 对应的 16 进制 32352E33
ble.indicate(0xFFF1, 4, "32352E33")
```

7 MTU 与连接参数操作

7.1. exchange_mtu

7.1.1. 接口概述

发起 MTU 交换请求。只有在 BLE 建立连接后才能发起交互请求。

7.1.2. 函数原型

ble.exchange_mtu(mtu)

7.1.3. 参数描述

参数名	是否必填	数据类型	默认值	说明
mtu	是	int	-	期望协商的 MTU 值(23~247)

7.1.4. 返回值描述

无返回值。 若执行失败，则抛出异常。

7.1.5. 示例

ble.exchange_mtu(220)

8 回调事件说明

当调用 ble.init(callback) 注册回调后，底层 BLE 事件会通过调度机制回调到 Python 层。 回调函数接收一个字典对象 event_dict，其中至少包含以下字段：

字段名	说明
event	事件类型，参考 9.6 事件常量

根据不同事件类型，还可能附带其他字段。

8.1. 连接事件

8.1.1. 事件类型

- BLE.EVT_CONNECTED
- BLE.EVT_DISCONNECTED

8.1.2. 回调数据格式

```
{
    "event": event_id
}
```

8.1.3. 示例

```
def ble_cb(evt):
    if evt["event"] == quectel.BLE.EVT_CONNECTED:
        print(evt)
```

8.2. MTU 事件

8.2.1. 事件类型

- BLE.EVT_MTU

8.2.2. 回调数据格式

```
{
    "event": event_id,
    "mtu": mtu_value
}
```

8.2.3. 示例

```
def ble_cb(evt):
    if evt["event"] == quectel.BLE.EVT_MTU:
        print("MTU 更新为:", evt["mtu"])
```

8.3. Characteristic 数据事件

8.3.1. 事件类型

- BLE.EVT_VAL_DATA

8.3.2. 回调数据格式

```
{
    "event": event_id,
    "uuid": uuid,
    "len": value_len,
    "value": value
}
```

8.3.3. 字段说明

字段名	说明
uuid	对应 Characteristic UUID
len	数据长度
value	收到的数据内容

8.3.4. 示例

```
def ble_cb(evt):
    if evt["event"] == quectel.BLE.EVT_VAL_DATA:
        print("收到 Characteristic 数据")
        print("UUID:", evt["uuid"])
        print("LEN :", evt["len"])
        print("VAL :", evt["value"])
```

8.4. Descriptor 数据事件

8.4.1. 事件类型

- BLE.EVT_DESCDATA

8.4.2. 回调数据格式

```
{
    "event": event_id,
    "uuid": char_uuid,
    "desc_uuid": desc_uuid,
    "len": value_len,
    "value": value
}
```

8.4.3. 字段说明

字段名	说明
uuid	**Descriptor 所属 Characteristic UUID**
desc_uuid	**Descriptor UUID**
len	数据长度
value	收到的数据内容（字符串）

8.4.4. 示例

```
def ble_cb(evt):
    if evt["event"] == quectel.BLE.EVT_DESCDATA:
        print("收到 Descriptor 数据:", evt)
```

8.5. 回调完整示例

```
import quectel

def ble_event_cb(event):
    evt = event.get("event")

    if evt == quectel.BLE.EVT_CONNECTED:
        print("BLE 已连接")
    elif evt == quectel.BLE.EVT_DISCONNECTED:
        print("BLE 已断开")
    elif evt == quectel.BLE.EVT_MTU:
        print("MTU:", event.get("mtu"))
    elif evt == quectel.BLE.EVT_VAL_DATA:
        print("VAL_DATA:", event)
    elif evt == quectel.BLE.EVT_DESCDATA:
        print("DESCDATA:", event)
    else:
        print("未知事件:", event)

ble = quectel.BLE()
ble.init(ble_event_cb)
```

9 常量定义

9.1. 广播类型常量

常量名	值	说明
BLE.ADV_CONNECTABLE_UNDIRECTED	**0**	可连接非定向广播
BLE.ADV_CONNECTABLE_DIRECTED_HIGH	**1**	可连接定向广播（高占空比）
BLE.ADV_NONCONNECTABLE_UNDIRECTED	**2**	不可连接非定向广播
BLE.ADV_SCANNABLE_UNDIRECTED	**3**	可扫描非定向广播

BLE.ADV_CONNECTABLE_DIRECTED_LOW	**4**	可连接定向广播（低占空比）

9.2. 地址类型常量

常量名	值	说明
BLE.ADDR_PUBLIC	0	公共地址
BLE.ADDR_RANDOM	1	随机地址

9.3. Character 属性常量

常量名	值	说明
BLE.PROP_BROADCAST	1	广播属性
BLE.PROP_READ	2	读属性
BLE.PROP_WRITE_NO_RESP	4	写（无响应）属性
BLE.PROP_WRITE	8	写属性
BLE.PROP_NOTIFY	16	Notify 属性
BLE.PROP_INDICATE	32	Indicate 属性
BLE.PROP_AUTH_SIGN_WRITE	64	签名写属性
BLE.PROP_EXT	128	扩展属性

9.4. 权限常量

常量名	值	说明
BLE.PERM_READ	1	可读
BLE.PERM_WRITE	2	可写
BLE.PERM_READ_AUTH	4	认证读
BLE.PERM_READ_AUTHOR	8	授权读
BLE.PERM_READ_ENC	16	加密读
BLE.PERM_READ_AUTHOR_AUTH	32	授权+认证读
BLE.PERM_WRITE_AUTH	64	认证写
BLE.PERM_WRITE_AUTHOR	128	授权写
BLE.PERM_WRITE_ENC	256	加密写
BLE.PERM_WRITE_AUTHOR_AUTH	512	授权+认证写

9.5. 数据格式常量

常量名		说明
BLE.DATAFMT_HEX	1	十六进制字符串模式
BLE.DATAFMT_STRING	2	普通字符串模式

9.6. 事件常量

常量名		说明
BLE.EVT_CONNECTED	0	BLE 已连接
BLE.EVT_DISCONNECTED	1	BLE 已断开
BLE.EVT_MTU	2	MTU 交换完成
BLE.EVT_DESCDATA	3	收到 Descriptor 数据
BLE.EVT_VAL_DATA	4	收到 Characteristic 数据